



Section 1: Type Annotations in TypeScript



What is a Type Annotation?

Type annotations let you explicitly specify the type of a variable, function parameter, or return value.

```
let message: string = "Hello, TypeScript!";
```

◆ Why Use Type Annotations?

- Catches errors early (at compile time).
- Improves code readability and maintainability.
- Helps with autocompletion in IDEs.



Basic Types

1. number

```
let count: number = 10;
```

2. string

```
let username: string = "Alice";
```

3. boolean

```
let isLoggedIn: boolean = true;
```

4. any (Allows any type – use cautiously!)

```
let data: any = "Could be anything";  
data = 42;
```

5. unknown (Like any, but safer)

```
let input: unknown = "Hello";
if (typeof input === "string") {
  console.log(input.toUpperCase());
}
```

6. void (For functions that don't return a value)

```
function logMessage(): void {
  console.log("Logging...");
}
```

7. never (For functions that never return – advanced, but worth a mention)

```
function throwError(): never {
  throw new Error("Something went wrong!");
}
```

◆ Arrays and Tuples

Arrays

```
let scores: number[] = [90, 80, 70];
let names: string[] = ["Alice", "Bob"];
```

Tuples

A tuple is an array with fixed number and types of elements.

```
let user: [string, number] = ["Alice", 25];
```

◆ Type Inference

TypeScript can infer the type even without a type annotation.

```
let city = "New York"; // inferred as string
```

But it's best to annotate explicitly for clarity, especially in Angular apps.

✅ Practice Exercise Suggestions

1. Declare variables for a user profile: name, age, isActive.
2. Create an array of product names.
3. Define a tuple for a coordinate: [latitude, longitude]

🧠 Section 2: Functions in TypeScript

Functions are a core part of TypeScript and Angular. Understanding how to properly type them helps prevent bugs and improves developer experience.

📌 Basic Function Syntax with Type Annotations

```
function greet(name: string): string {  
    return `Hello, ${name}`;  
}
```

- name: string → parameter type
- : string → return type

◆ Typing Function Parameters

You can specify the type for each parameter.

```
function add(x: number, y: number): number {  
    return x + y;  
}
```

◆ Optional Parameters

Use `?` to mark a parameter as optional.

```
function greet(name?: string): string {  
  return name ? `Hello, ${name}` : "Hello!";  
}
```

Note: Optional parameters must come **after** required ones.

◆ Default Parameters

You can assign default values to parameters.

```
function greet(name: string = "Guest"): string {  
  return `Welcome, ${name}`;  
}
```

◆ Typing the Return Value

Always specify the return type explicitly, especially in larger codebases like Angular apps.

```
function isEven(num: number): boolean {  
  return num % 2 === 0;  
}
```

◆ Arrow Functions

Arrow functions are often used in Angular components and services.

```
const multiply = (a: number, b: number): number => {  
  return a * b;  
};
```

Or more concisely:

```
const square = (n: number): number => n * n;
```

◆ Function Type Aliases

You can define a function type and reuse it.

```
type MathOp = (a: number, b: number) => number;
```

```
const divide: MathOp = (x, y) => x / y;
```

✓ Practice Exercise Suggestions

1. Write a function to check if a string includes the word "TypeScript".
2. Create a function with one required and one optional parameter.
3. Define a type alias for a function that takes a string and returns void.

🧠 Section 3: Interfaces and Types in TypeScript

📌 What are Interfaces and Type Aliases?

Both are used to define the **shape of an object** (its properties and types), but each has slightly different capabilities.

◆ Interfaces

```
interface User {  
  name: string;  
  age: number;  
  isActive: boolean;  
}
```

You can use it to type-check variables:

```
const user1: User = {  
  name: "Alice",  
  age: 25,  
  isActive: true,  
};
```

◆ Optional Properties

Use `?` to mark a property as optional.

```
interface Product {  
  name: string;  
  price: number;  
  description?: string;  
}
```

◆ Readonly Properties

Use `readonly` to prevent modifications after initialization.

```
interface Config {  
  readonly apiUrl: string;  
}
```

◆ Interface Extension

You can extend interfaces to build upon existing types.

```
interface Person {  
  name: string;  
}  
  
interface Employee extends Person {  
  employeeId: number;  
}
```

◆ Type Aliases

```
type Point = {  
  x: number;  
  y: number;  
};
```

Used just like an interface:

```
const pt: Point = { x: 10, y: 20 };
```

◆ Union and Intersection with Types

Only `type` can do these:

```
type Status = "success" | "error" | "loading";  
  
type A = { a: string };  
type B = { b: number };  
type AB = A & B;
```

◆ When to Use interface vs type ?

Feature	interface	type
Extendable	✓ Yes	✓ Yes (with &)
Readable in IntelliSense	✓ Yes	✓ Yes
Unions and Intersections	✗ No	✓ Yes
Angular preference	✓ Often used	✓ Also valid

✓ Use **interfaces** when defining models, component props, or service response types.

✓ Use **types** for union types, utility compositions, or when flexibility is needed.

✓ Practice Exercise Suggestions

1. Create an interface for a blog post with `title` , `body` , and optional `author` .
2. Define a type alias for a position object with `x` and `y` .
3. Extend an interface `Animal` with another interface `Dog` .

🧠 Section 4: Classes in TypeScript

📌 What is a Class?

A class is a template for creating objects that share structure (properties) and behavior (methods).

```
class Person {  
  name: string;  
  age: number;  
  
  constructor(name: string, age: number) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet(): string {  
    return `Hello, my name is ${this.name}`;  
  }  
}
```

◆ Creating an Object from a Class

```
const person1 = new Person("Alice", 28);  
console.log(person1.greet()); // Hello, my name is Alice
```

◆ Access Modifiers

Modifier	Description
public	(default) Accessible from anywhere
private	Accessible only inside the class
protected	Accessible inside class and subclasses

```
class BankAccount {  
    private balance: number;  
  
    constructor(initialBalance: number) {  
        this.balance = initialBalance;  
    }  
  
    deposit(amount: number): void {  
        this.balance += amount;  
    }  
  
    getBalance(): number {  
        return this.balance;  
    }  
}
```

◆ Readonly Properties

You can mark a property as `readonly` to prevent changes after initialization.

```
class Config {  
    readonly appName: string = "MyApp";  
}
```

◆ Static Members

Static members belong to the class itself, not to instances.

```
class MathUtils {  
    static PI: number = 3.14159;  
  
    static square(n: number): number {  
        return n * n;  
    }  
}  
  
console.log(MathUtils.square(4)); // 16
```

◆ Inheritance

You can create a class based on another class using `extends`.

```
class Animal {  
    constructor(public name: string) {}  
  
    makeSound(): string {  
        return `${this.name} makes a sound.`;  
    }  
}  
  
class Dog extends Animal {  
    bark(): string {  
        return `${this.name} barks.`;  
    }  
}  
  
const dog = new Dog("Buddy");  
console.log(dog.bark()); // Buddy barks.
```

◆ Parameter Properties (Shortcut)

TypeScript lets you define and initialize properties directly in the constructor.

```
class User {
  constructor(public username: string, private password: string) {}

  getUsername(): string {
    return this.username;
  }
}
```

✓ Practice Exercise Suggestions

1. Create a `Book` class with properties: `title`, `author`, and a method `printInfo()`.
2. Create a `Rectangle` class with properties `width`, `height` and a method `getArea()`.
3. Extend a base class `Vehicle` with a subclass `Car` that adds a `drive()` method.

Section 5: Modules and Imports/Exports

Modules help organize code by splitting it into multiple files. In TypeScript (and modern JavaScript), every file is its own **module** by default.

◆ Exporting

Use the `export` keyword to expose functions, classes, interfaces, variables, etc., from a module.

Exporting individually:

```
// mathUtils.ts
export function add(x: number, y: number): number {
  return x + y;
}

export const PI = 3.14159;
```

Exporting together:

```
// user.ts
function login(username: string): void {
  console.log("Logging in:", username);
}

function logout(): void {
  console.log("Logging out...");
}

export { login, logout };
```

Default Export:

Each module can have **one** default export.

```
// logger.ts
export default function log(message: string): void {
  console.log("LOG:", message);
}
```

◆ Importing

Importing named exports:

```
// main.ts
import { add, PI } from './mathUtils';

console.log(add(2, 3)); // 5
console.log(PI); // 3.14159
```

Importing with alias:

```
import { add as sum } from './mathUtils';
```

Importing all as an object:

```
import * as MathUtils from './mathUtils';  
  
console.log(MathUtils.add(1, 2));
```

Importing default exports:

```
import log from './logger';  
  
log("Hello!");
```

Why Use Modules?

- Keep code organized and reusable
- Avoid naming conflicts
- Enable lazy loading and performance optimization
- Essential for working in larger projects

Practice Exercise Suggestions

1. Create a file `calculator.ts` with `add`, `subtract`, and `multiply` functions. Export them.
2. Import those functions in `main.ts` and use them.
3. Try using both named and default exports.

Section 6: Arrays and Tuples in TypeScript

Arrays and Tuples are both used to store multiple values. However, they differ in the types of data they store and their intended use.

◆ Arrays

Arrays are used to store lists of values of the **same type**.

Declaring an Array:

```
let numbers: number[] = [1, 2, 3, 4];  
let strings: string[] = ['apple', 'banana', 'cherry'];
```

Or using the generic `Array` type:

```
let numbers: Array<number> = [1, 2, 3, 4];
```

Accessing Array Elements:

```
let firstNumber = numbers[0]; // 1  
let firstString = strings[0]; // 'apple'
```

Array Methods:

You can use typical array methods like `push()`, `pop()`, `map()`, `filter()`, etc.

```
numbers.push(5); // Adds 5 at the end  
numbers.map(n => n * 2); // [2, 4, 6, 8, 10]
```

◆ Tuple

Tuples are like arrays, but they allow **heterogeneous** types (different data types in a single collection).

Declaring a Tuple:

```
let person: [string, number] = ['Alice', 30];
```

In this case, `person` can only have a **string** as the first element and a **number** as the second element.

Accessing Tuple Elements:

```
let name = person[0]; // 'Alice'
let age = person[1];  // 30
```

Tuple with Optional Elements:

You can have optional elements in a tuple by using `?`.

```
let userInfo: [string, number?, boolean?] = ['Bob', 25];
```

This tuple can have an optional `number` (age) and an optional `boolean` (active status).

◆ Arrays vs. Tuples

- **Arrays:** All elements must have the same type.
- **Tuples:** Elements can have different types, but their positions are fixed.

✓ Practice Exercise Suggestions

1. Create an array of numbers and find the sum of all the elements.
2. Create a tuple representing a book with `title` (string), `author` (string), and `year` (number).
3. Define a function that accepts a tuple with two elements, `name` (string) and `age` (number), and returns a greeting message.

🧠 Section 7: Generics in TypeScript

📌 What Are Generics?

Generics allow you to write functions, classes, and interfaces that can work with **any data type**, while still maintaining type safety. They provide a way to write **flexible, reusable code** that can work with a variety of types.

◆ Generic Functions

A generic function is defined with a type parameter, which is specified by a placeholder (like `T` or `U`) rather than a concrete type.

```
function identity<T>(value: T): T {  
    return value;  
}
```

Here, `T` is a type parameter that will be replaced by a specific type when the function is called.

Calling the Generic Function:

```
let result1 = identity<string>('Hello'); // result1 is of type string  
let result2 = identity<number>(123);    // result2 is of type number
```

If you don't specify a type, TypeScript will infer the type based on the argument passed:

```
let result3 = identity(42); // inferred as number
```

◆ Generic Arrays

You can use generics to define arrays that can hold any type of elements.

```
function printArray<T>(arr: T[]): void {  
    arr.forEach(item => console.log(item));  
}
```

Example:

```
printArray([1, 2, 3]); // Array of numbers  
printArray(['apple', 'banana']); // Array of strings
```

◆ Generic Interfaces

You can create interfaces that are **generic**, allowing them to work with different types.

```
interface Box<T> {  
  value: T;  
}  
  
let box1: Box<number> = { value: 42 };  
let box2: Box<string> = { value: 'hello' };
```

◆ Generic Classes

Classes can also use generics to handle different data types.

```
class Container<T> {  
  constructor(private value: T) {}  
  
  getValue(): T {  
    return this.value;  
  }  
}
```

Example:

```
let stringContainer = new Container('hello');  
console.log(stringContainer.getValue()); // 'hello'  
  
let numberContainer = new Container(42);  
console.log(numberContainer.getValue()); // 42
```

◆ Constraints on Generics

Sometimes, you might want to restrict the types that can be used with generics. This is where **constraints** come in.

```
function getLength<T extends { length: number }>(item: T): number {  
    return item.length;  
}
```

Example:

```
getLength('hello'); // Works, because strings have a length  
getLength([1, 2, 3]); // Works, because arrays have a length  
getLength(123); // Error, numbers don't have a length
```

◆ Default Generic Types

You can also provide default values for generics, so if no type is specified, TypeScript will use the default type.

```
function wrapInArray<T = string>(value: T): T[] {  
    return [value];  
}
```

```
console.log(wrapInArray(42)); // [42] (type inferred as number[])  
console.log(wrapInArray('test')); // ['test'] (type inferred as string[])
```

✓ Practice Exercise Suggestions

1. Create a generic `swap` function that swaps the position of two elements in an array.
2. Create a generic class `Queue` that holds items of any type and provides methods to enqueue and dequeue items.
3. Write a function that takes a generic parameter `T` and checks whether the given object has a `length` property, using a constraint.



Section 8: Type Aliases and Unions in TypeScript

◆ Type Aliases

Type aliases are a way to give a name to a **complex type**, making the code more readable and reusable. You can define type aliases for primitive types, objects, or even more complex combinations.

Defining a Type Alias:

```
type Person = {  
  name: string;  
  age: number;  
};
```

Now, you can use `Person` as a type throughout your code:

```
const user: Person = {  
  name: 'Alice',  
  age: 30,  
};
```

Type Aliases for Function Types:

You can also use type aliases for functions:

```
type AddFunction = (a: number, b: number) => number;  
  
const add: AddFunction = (a, b) => a + b;
```

◆ Union Types

A **Union Type** allows a value to be one of several types. You define a union by separating the types with a pipe (`|`).

Defining a Union Type:

```
let value: string | number;  
value = 'Hello';  
value = 42;
```

In this case, `value` can either be a `string` or a `number`.

Union Types with Functions:

```
function printId(id: string | number): void {  
  console.log(`The ID is: ${id}`);  
}
```

Union with Arrays:

You can use union types in arrays as well:

```
let list: (string | number)[] = ['apple', 42, 'banana', 100];
```

◆ Type Aliases with Union Types

Type aliases and union types can be used together to create more complex and flexible types.

```
type StringOrNumber = string | number;  
  
let value: StringOrNumber;  
value = 'hello'; // OK  
value = 100;     // OK  
value = true;    // Error: boolean is not allowed
```

◆ Type Guards with Union Types

When working with union types, it's often necessary to **narrow down** the type of a variable. This can be done using **type guards** (such as `typeof` or `instanceof`).

Using typeof :

```
function printLength(value: string | number): void {
  if (typeof value === 'string') {
    console.log(value.length); // OK, only string has length
  } else {
    console.log('Number does not have length');
  }
}
```

Using instanceof :

```
class Person {
  constructor(public name: string) {}
}

class Dog {
  constructor(public breed: string) {}
}

function printInfo(animal: Person | Dog): void {
  if (animal instanceof Person) {
    console.log(`Person: ${animal.name}`);
  } else if (animal instanceof Dog) {
    console.log(`Dog breed: ${animal.breed}`);
  }
}
```

◆ Intersection Types (Bonus)

While not strictly part of Type Aliases and Unions, **Intersection Types** allow you to combine multiple types into one. This is useful for merging multiple types or ensuring that an object conforms to several structures.

```
type Employee = {
  name: string;
  role: string;
};

type Contact = {
  email: string;
  phone: string;
};

type EmployeeWithContact = Employee & Contact;

const employee: EmployeeWithContact = {
  name: 'Alice',
  role: 'Developer',
  email: 'alice@example.com',
  phone: '123-456-7890',
};
```

✅ Practice Exercise Suggestions

1. Create a `Shape` type alias that can be either a `circle` or a `rectangle`, with properties specific to each shape (e.g., `radius` for circle and `width / height` for rectangle).
2. Write a function `isAdult` that takes a `string | number` as an argument and returns whether the value represents an adult age.
3. Create a `UserInfo` type alias using an intersection of two types: `User` (with `name` and `age`) and `Address` (with `city` and `zipcode`).

🧠 Section 9: Advanced Types in TypeScript

◆ Literal Types

Literal types are exact values a variable can hold, instead of general types like `string` or `number`. You can define a variable to accept a specific string, number, or boolean value.

Example with String Literals:

```
let direction: 'left' | 'right'; // direction can only be 'left' or 'right'

direction = 'left'; // Valid
direction = 'right'; // Valid
direction = 'up';    // Error: 'up' is not a valid value
```

Example with Number Literals:

```
let code: 404 | 200; // code can only be 404 or 200

code = 200; // Valid
code = 500; // Error: 500 is not allowed
```

This is useful when you want to restrict a variable to a very specific set of values.

◆ Index Types and Lookup Types

In TypeScript, **Index Types** help you extract types from a given object or array. **Lookup Types** allow you to look up the type of a property based on the key of an object.

Example with Index Type:

```
type Person = {
  name: string;
  age: number;
};

type PersonKey = keyof Person; // 'name' | 'age' (the keys of Person)

let key: PersonKey;
key = 'name'; // Valid
key = 'age';  // Valid
key = 'address'; // Error: 'address' is not a key of Person
```

Lookup Types:

```
type Person = {  
  name: string;  
  age: number;  
};  
  
type NameType = Person['name']; // string
```

In this case, `NameType` is just a `string`, because it's the type of `name` in the `Person` object.

◆ Conditional Types

Conditional types allow you to choose between types based on a condition. They follow the syntax:

```
T extends U ? X : Y
```

- If `T` is assignable to `U`, the result is `X`.
- Otherwise, the result is `Y`.

Example of Conditional Types:

```
type IsString<T> = T extends string ? 'Yes' : 'No';  
  
type Test1 = IsString<string>; // 'Yes'  
type Test2 = IsString<number>; // 'No'
```

Conditional Types with `infer`:

The `infer` keyword allows you to capture types within a conditional type.

```
type ReturnTypeOf<T> = T extends (...args: any[]) => infer R ? R : never;  
  
type FunctionReturnType = ReturnTypeOf<() => string>; // string
```

◆ Mapped Types

Mapped types allow you to create a new type by transforming the properties of an existing type.

Example with Mapped Types:

```
type Readonly<T> = {
  readonly [P in keyof T]: T[P];
};

type Person = {
  name: string;
  age: number;
};

type ReadonlyPerson = Readonly<Person>;
// ReadonlyPerson is { readonly name: string, readonly age: number }
```

In this example, `Readonly<T>` takes any type `T` and makes all of its properties `readonly`.

Another Example:

```
type Optional<T> = {
  [P in keyof T]?: T[P];
};

type Person = {
  name: string;
  age: number;
};

type OptionalPerson = Optional<Person>;
// OptionalPerson is { name?: string, age?: number }
```

◆ Type Guards with `is`

A **Type Guard** is a way to narrow down the type of a variable inside a conditional block. Using the `is` keyword, you can create custom type guards.

Example:

```
class Dog {
  bark() {
    console.log('Woof');
  }
}

class Cat {
  meow() {
    console.log('Meow');
  }
}

function isDog(animal: Dog | Cat): animal is Dog {
  return (animal as Dog).bark !== undefined;
}

const animal: Dog | Cat = new Dog();

if (isDog(animal)) {
  animal.bark(); // TypeScript knows that `animal` is a `Dog` here
} else {
  animal.meow(); // TypeScript knows that `animal` is a `Cat` here
}
```

◆ Template Literal Types (Template Strings)

Template literal types allow you to define string types that are based on the structure of other string values.

```
type Greeting = `Hello, ${string}`;

let message: Greeting = 'Hello, Alice'; // Valid
message = 'Hi, Alice'; // Error: 'Hi, Alice' doesn't match the template
```

This allows more complex string patterns in your types.

Practice Exercise Suggestions

1. Create a type alias for a `Color` that can be either `'red'`, `'blue'`, or `'green'`.
2. Write a conditional type `IsNumber<T>` that returns `'Yes'` if `T` is a number, otherwise `'No'`.
3. Create a mapped type `Writable<T>` that converts all properties of a given type `T` to writable (removes `readonly`).
4. Implement a type guard `isEven` that checks if a given number is even, returning `true` if it is.